Step 3 — A Research Agent That Looks Things Up Before Anyone Else Works — Solution

Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 3: everything downstream is only as true as the research it starts from. The Research agent **looks facts up with a tool first**, then turns what it found into notes — and when the tool finds nothing, it says so instead of inventing. It's built and tested alone, like the Writer. **If not:** the article would be built on whatever the model happened to remember about the topic.

After this step the project runs from a topic: research → write → review. Files not shown here don't change.

The tool and its data

Story — `data/sample/facts.json` + `tools/lookup.py`: a small local fact store with sources, behind a Doc06 `@tool`. Test the tool alone first: then, if a note looks wrong later, you already know whether it's the tool or the agent. (Also create an empty `src/tools/__init__.py`.) **If not:** you'd be debugging the tool and the agent's use of it at the same time.

`data/sample/facts.json`

```
{
  "electric bikes": [
    "Europe sold over 5 million e-bikes in 2022. [market-report]",
    "Most e-bikes only help while you pedal. [bike-guide]",
    "A battery gives 40 to 100 km per charge. [battery-review]",
    "EU e-bike motors are limited to 25 km/h. [eu-rules]",
    "E-bike commutes cut emissions by over 90 percent. [transport-study]"
  ],
  "remote work": [
    "28 percent of US work days were remote in 2023. [wfh-survey]",
    "Remote workers save 72 minutes a day of commuting. [commute-study]",
    "Many firms use 2 or 3 office days a week. [hybrid-report]",
    "Loneliness is the top downside of remote work. [wellbeing-survey]"
  ],
  "solar panels": [
    "Home solar panels last 25 to 30 years. [solar-guide]",
    "Panel prices fell about 90 percent from 2010 to 2020. [price-report]",
```

```
    "Sunny-region systems pay back in 6 to 10 years. [payback-study]",
    "Panels still make power on cloudy days. [solar-basics]"
  ]
}
```

```
# src/tools/lookup.py
# The Research agent's lookup tool. load_facts() reads facts.json;
# lookup_facts(topic) returns one fact per line for the first known topic
# found in the text, or "" when the topic is unknown.
import json

from langchain_core.tools import tool

FACTS_PATH = "data/sample/facts.json"

def load_facts() -> dict:
    with open(FACTS_PATH) as facts_file:
        return json.loads(facts_file.read())

@tool
def lookup_facts(topic: str) -> str:
    """Look up known facts, with sources, for a topic. Returns one fact
    per line, or an empty string if the topic is not known."""
    facts = load_facts()
    lowered = topic.lower()
    for known_topic in facts:
        if known_topic in lowered:
            text = ""
            for fact in facts[known_topic]:
                text = text + "- " + fact + "\n"
            return text
    return ""
```

Back to this step · Hint 1 · Hint 2 · Solution

Solution — look up first, then write notes from what was found

Story — `prompts/research.txt`: turn the looked-up facts into notes: one line per fact, keep the `[source]` tag, add nothing. **If not:** the model could "improve" the notes with extra facts nobody looked up.

`src/prompts/research.txt`

```
You are the Researcher on a content team. Turn the looked-up facts below
into research notes for the topic "{topic}": one line per fact, in plain
words, each ending with its [source] tag. Do not add any fact that is
not listed.

Looked-up facts:
{facts}
```

Story — `agents/research_agent.py`: `lookup_facts` runs first. If it returns nothing, `run_research()` returns `""` and logs a warning — no model call, no invented notes. **If not:** an unknown topic would still produce an article.

```
# src/agents/research_agent.py
# The Research agent. run_research(topic) looks facts up with the tool
# FIRST; if nothing is found it returns "", otherwise an LCEL chain turns
# the facts into notes.
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate

from src.models.llm import get_model
from src.tools.lookup import lookup_facts
from src.utils.logger import get_logger
from src.utils.prompts import load_prompt

def run_research(topic: str) -> str:
    logger = get_logger(__name__)
    facts = lookup_facts.invoke({"topic": topic})
    if facts == "":
        logger.warning(f"no facts found for {topic!r}")
        return ""
    prompt = ChatPromptTemplate.from_template(load_prompt("research"))
```

```
    chain = prompt | get_model() | StrOutputParser()
    return chain.invoke({"topic": topic, "facts": facts})
```

Changed from Step 2: `main.py` asks for a topic, runs Research, stops honestly when nothing is found, and hands the notes to the same `write_with_review()` loop.

```
# main.py
# The file you run: python main.py
# Step 3 version: ask for a topic, run Research, stop if nothing is found,
# then write and review from the notes.
import sys

from src.agents.research_agent import run_research
from src.agents.reviewer_agent import PASS_SCORE, run_reviewer
from src.agents.writer_agent import run_writer
from src.config import load_config
from src.exceptions import MissingConfigError
from src.utils.logger import get_logger

MAX_REVISIONS = 3

def write_with_review(findings: str) -> dict:
    logger = get_logger(__name__)
    result = {
        "draft": "",
        "accepted": False,
        "feedback_history": [],
        "revision_count": 0,
    }
    feedback = ""
    for round_number in range(1, MAX_REVISIONS + 1):
        result["draft"] = run_writer(findings, feedback)
        verdict = run_reviewer(findings, result["draft"])
        logger.info(f"round {round_number}: score {verdict.score}")
        if verdict.score >= PASS_SCORE:
            result["accepted"] = True
            return result
        feedback = verdict.feedback
        result["feedback_history"].append(feedback)
        result["revision_count"] = result["revision_count"] + 1
    return result

def print_report(result: dict) -> None:
    if result["accepted"]:
        print("\nFinal article:\n")
        print(result["draft"])
```

```
    else:
        attempts = result["revision_count"]
        print(f"\nCouldn't agree after {attempts} revisions. Feedback:")
        number = 1
        for feedback in result["feedback_history"]:
            print(f"  round {number}: {feedback}")
            number = number + 1
        print("\nLast draft (NOT approved):\n")
        print(result["draft"])

def main() -> None:
    try:
        load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    # new in Step 3: a topic in, real research notes instead of samples
    topic = input("Topic: ").strip()
    notes = run_research(topic)
    if notes == "":
        print("No article: nothing is known about this topic yet.")
        return
    print(f"Research notes:\n{notes}\n")
    result = write_with_review(notes)
    print_report(result)

if __name__ == "__main__":
    main()
```

The Step 3 tests

Story — `tests/unit/test_lookup.py` **:** the tool alone, no API key — a known topic returns its facts, an unknown one returns `""`. **If not:** a broken topic match would look like a bad model.

```
# tests/unit/test_lookup.py
# Checks the Research agent's lookup tool with no API key. Each case prints
# OK or WRONG.
# Run from the repo root: python -m tests.unit.test_lookup
from src.tools.lookup import lookup_facts


def test_known_topic() -> None:
    facts = lookup_facts.invoke({"topic": "Why Electric Bikes are popular"})
    lines = facts.strip().split("\n")
```

```
    result = "OK"
    if len(lines) != 5:
        result = "WRONG"
    print(f"case 1 (known topic -> 5 facts) -> {result}")


def test_unknown_topic() -> None:
    facts = lookup_facts.invoke({"topic": "deep sea mining"})
    result = "OK"
    if facts != "":
        result = "WRONG"
    print(f"case 2 (unknown topic -> empty) -> {result}")


if __name__ == "__main__":
    test_known_topic()
    test_unknown_topic()
```

Changed from Step 2: the integration test adds case 3 (a known topic keeps its sources) and case 4 (an unknown topic gives `""`).

```
# tests/integration/test_live_agents.py
# Runs the agents against the real OpenAI API — each step adds cases.
# Each case prints OK or WRONG. Needs a valid OPENAI_API_KEY in .env.
# Run from the repo root: python -m tests.integration.test_live_agents
from src.agents.research_agent import run_research
from src.agents.reviewer_agent import PASS_SCORE, run_reviewer
from src.agents.writer_agent import run_writer
from src.config import load_config

FINDINGS = (
    "Range:\n- A battery gives 40 to 100 km per charge. [battery-review]\n"
    "Rules:\n- EU e-bike motors are limited to 25 km/h. [eu-rules]\n"
)


def test_writer_keeps_sources() -> None:
    draft = run_writer(FINDINGS)
    result = "OK"
    if "[" not in draft:
        result = "WRONG"
    print(f"case 1 (writer keeps sources) -> {result}")


def test_reviewer_rejects_weak_draft() -> None:
    verdict = run_reviewer(FINDINGS, "E-bikes are good.")
```

```
    result = "OK"
    if verdict.score >= PASS_SCORE:
        result = "WRONG"
    print(f"case 2 (weak draft fails)     -> {result}")


def test_research_keeps_sources() -> None:
    notes = run_research("electric bikes")
    result = "OK"
    if notes.count("[") < 5:
        result = "WRONG"
    print(f"case 3 (known topic sourced)  -> {result}")


def test_research_unknown_topic() -> None:
    notes = run_research("deep sea mining")
    result = "OK"
    if notes != "":
        result = "WRONG"
    print(f"case 4 (unknown topic empty)  -> {result}")


if __name__ == "__main__":
    load_config()
    test_writer_keeps_sources()
    test_reviewer_rejects_weak_draft()
    test_research_keeps_sources()
    test_research_unknown_topic()
```

Run everything (from the repo root):

```
python -m tests.unit.test_lookup
python -m tests.integration.test_live_agents
python main.py
```

Expected unit output:

```
case 1 (known topic -> 5 facts) -> OK
case 2 (unknown topic -> empty) -> OK
```